Context-Aware IoT-Based Field Monitoring and Fertilization Guidance for Sri Lankan Paddy Cultivation

Karalliyadda K K U B M U B

(IT22178718)

Individual Component: Real time IoT field Monitoring and fertilization prediction

Group ID: R26-IT-066

Submitted in Partial Fulfilment of the Requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

Supervisors: Ms. Chathurangika Kahandawaarachchi | Ms. Buddhima Attanayaka

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix C). I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Student name: Karalliyadda K K U B M U B Student ID: IT22178718

Signature:  Date: 2026/10/11

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Supervisor certification record.

Role	Name	Signature	Date
Supervisor	Ms. Chathurangika Kahandawaarachchi		
Co-supervisor	Ms. Buddhima Attanayaka		

ABSTRACT

Sri Lankan paddy farmers usually apply fertilizer on a fixed calendar, not on the measured condition of their soil, which wastes fertilizer in some fields and under-feeds others. This thesis documents the component of the R26-IT-066 smart paddy decision support system that turns field soil readings into a fertilization decision: whether fertilizer is needed now and, if so, how much in kg/ha. The component was implemented as an ESP32-based soil station that publishes moisture, temperature, humidity and N, P and K readings to a Firebase Realtime Database, a Flutter field screen that synchronises those readings every 5 seconds, and a Flask prediction service that runs a two-stage model: a random forest classifier decides whether fertilizer is needed and an XGBoost regressor estimates the amount. Both models were trained on a 24,000-record field dataset with a 20 per cent held-out test set. The classifier reached 0.9400 accuracy on 4,800 held-out records, with precision 0.9991 and recall 0.9208 for the fertilizer-needed class: it issued 3 unnecessary recommendations and missed 285 real needs. The regressor explained 86.7 per cent of the variance in fertilizer amount (R^2 0.8666). Nitrogen, phosphorus and potassium carry 97.2 per cent of the classifier's importance, so moisture, pH, temperature, growth stage and variety barely influence the decision. Twelve behavioural scenarios matched the expected decision in all 12 cases, but the recommended amount does not follow the Department of Agriculture growth-stage schedule. A repository review found that two of the five growth stages sent by the mobile screen are rejected by the model's encoder, and that the telemetry store keeps only the latest reading. Both are reported as defects rather than hidden.

Keywords: precision agriculture, Internet of Things, fertilizer recommendation, random forest, XGBoost, paddy cultivation.

ACKNOWLEDGEMENTS

I thank my supervisor, Ms. Chathurangika Kahandawaarachchi and my co supervisor Ms. Buddhima Attanayaka for directing this research towards field-level fertilization guidance and for their guidance throughout the year. I thank my group members of R26-IT-066: IT22266446 (disease-aware yield estimation and financial decision support), IT21811104 (crop recommendation and market analysis) and IT22261632 (disease detection and severity analysis), whose modules consume the soil data this component produces. I thank the farmers who allowed the device to be installed in their field and who gave feedback on the application .Finally, I thank my family for their support throughout the final year.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

ABSTRACT ii

ACKNOWLEDGEMENTS iii

TABLE OF CONTENTS iv

LIST OF FIGURES viii

LIST OF TABLES ix

LIST OF ABBREVIATIONS xi

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 2

1.3 Problem Statement 3

1.4 Research Gap 4

1.5 Aim and Objectives 5

1.6 Research Questions or Hypotheses 5

1.7 Scope Limitations and Assumptions 6

1.8 Individual Contribution 7

1.9 Thesis Structure 7

2 LITERATURE REVIEW AND RELATED WORK 9

2.1 Review Method and Scope 9

2.2 Domain and Theoretical Foundations 9

2.3 Related Research 10

2.4 Existing Systems Products and Techniques 12

2.5 Gap Synthesis and Conceptual Framework 13

3 RESEARCH METHODOLOGY 15

3.1 Research Design 15
3.2 Data Participants or Study Materials 15
3.3 Experimental or Study Procedure 19
3.4 Measures and Evaluation Metrics 20
3.5 Data Analysis 21
3.6 Ethics Privacy Security and Responsible AI 22
3.7 Reproducibility and Research Artefacts 23
4 PRODUCT AND SOLUTION DESIGN 24
4.1 Requirements 24
4.2 Component Architecture Within the Complete Solution 25
4.3 Design Alternatives and Decisions 26
4.4 Detailed Design 27
4.5 Standards and Professional Practice 29
5 IMPLEMENTATION INTEGRATION AND TESTING 30
5.1 Development Environment and Technology Stack 30
5.2 Component Implementation 30
5.3 Integration with the Other Components 32
5.4 Verification and Software Testing 33
5.5 Deployment and Operation 34
6 RESULTS AND VALIDATION 36
6.1 Research Results 36
6.1.1 Protocol A: algorithm comparison on the baseline feature set 36
6.1.2 Protocol A: feature-group ablation 37
6.1.3 Protocol A: behavioural scenario tests 39
6.1.4 Protocol B: optimised deployed configuration 40
6.1.5 Feature importance and threshold sensitivity 45

6.2 Product and Technical Results 48
6.3 User or Stakeholder Validation 49
6.4 Integration Results 50
6.5 Summary of Findings 51
7 DISCUSSION 53
7.1 Interpretation of Findings 53
7.2 Comparison with Literature Baselines and Existing Solutions 54
7.3 Research Contribution 55
7.4 Product Contribution 55
7.5 Limitations and Threats to Validity 56
7.6 Implications 57
8 COMMERCIALISATION AND SUSTAINABILITY 59
8.1 Value Proposition and Target Users 59
8.2 Market and Competitive Position 59
8.3 Business or Sustainability Model 60
8.4 Scalability Intellectual Property and Compliance 60
8.5 Contribution to the Sustainable Development Goals 61
9 CONCLUSION AND FUTURE DIRECTIONS 62
9.1 Conclusion 62
9.2 Achievement of Objectives 62
9.3 Individual Research and Product Contributions 63
9.4 Future Directions 63
REFERENCES 65
GLOSSARY 68
APPENDICES 70
Appendix A - Extended Results 70

Appendix B - Academic Integrity and AI Use Disclosure 71

LIST OF FIGURES

Figure 1.1. Component within the Smart Camp Box data flow. 3
Figure 2.1. Conceptual framework for site-level flood risk. 14
Figure 3.1. Class distribution of the training dataset. 17
Figure 3.2. Correlation matrix of model inputs and target. 18
Figure 4.1. Layered architecture and owned layers. 25
Figure 4.2. Feature assembly for a single prediction. 27
Figure 5.1. End-to-end sequence of one prediction cycle. 32
Figure 6.1. Protocol A: held-out test metrics of four learners. 37
Figure 6.2. Protocol A: ROC-AUC change per removed feature 38
Figure 6.3. Protocol A: probabilities for ten scenarios. 40
Figure 6.4. Protocol B: confusion matrix at threshold 0.30. 42
Figure 6.5. Protocol B: ROC curve of the deployed artefact. 43
Figure 6.6. Protocol B: precision-recall curve 44
Figure 6.7. Protocol B: feature importance chart. 46
Figure 6.8. Threshold sensitivity of the configuration. 47
Figure 6.9. Protocol B: probability distributions by class. 48

LIST OF TABLES

Table 2.1. Existing systems versus component requirements. 12
Table 3.1. Attribute summary of the training dataset. 16
Table 3.2. Evaluation measures used in this study. 20
Table 4.1. Functional requirements for this component. 24
Table 4.2. Non-functional requirements and mechanisms. 24
Table 4.3. Design alternatives and decisions taken. 26
Table 4.4. Deployed feature vector and roles. 28
Table 4.5. HTTP surface exposed by this component. 28
Table 5.1. Technology stack and version pins 30
Table 5.2. Provider timeouts and failure behaviour. 31
Table 5.3. Integration points with the rest of the solution. 32
Table 5.4. Test suites and their outcomes. 33
Table 6.1. Protocol A: five-fold cross-validation results. 36
Table 6.2. Protocol A: held-out test results. 36
Table 6.3. Protocol A: single-feature ablation. 38
Table 6.4. Protocol A: scenario tests and SHAP contributions. 39
Table 6.5. Protocol B: deployed metrics at threshold 0.30. 41
Table 6.6. Protocol B: confusion matrix of the artefact. 41
Table 6.7. Recorded versus re-verified metrics. 44
Table 6.8. Protocol B: feature importance values. 45
Table 6.9. Threshold sensitivity at four thresholds. 47
Table 6.10. Live inference smoke test. 48
Table 6.11. Deployed model artefact metadata. 49
Table 7.1. Threats to validity and limitations 56
Table 8.1. Cost summary for the programme (LKR). 60
Table 9.1. Achievement of individual objectives. 62
Table A.1. Full threshold sweep (5,000 records) 70
Table A.2. Scenario outcomes: predicted versus expected. 71

LIST OF ABBREVIATIONS

Abbreviations used in this thesis, listed alphabetically.

Abbreviation	Meaning
API	Application Programming Interface
CM	Confusion matrix
DoA	Department of Agriculture, Sri Lanka
DSR	Design Science Research
DSS	Decision Support System
EC	Electrical conductivity
ESP32	Espressif 32-bit microcontroller with Wi-Fi
FN / FP	False negative / false positive
FR / NFR	Functional / non-functional requirement
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IRRI	International Rice Research Institute
JSON	JavaScript Object Notation
LR	Logistic regression
ML	Machine learning
NPK	Nitrogen, phosphorus and potassium
R^2	Coefficient of determination
RF	Random forest
RQ	Research question
RTDB	Firebase Realtime Database
SDG	Sustainable Development Goal
SO / IO	Specific objective (proposal) / individual objective (thesis)
TN / TP	True negative / true positive
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Overall Project Context

Paddy is Sri Lanka's staple crop, and how much a farmer earns from a season depends heavily on how well fertilizer is timed and dosed. Department of Agriculture (DoA) guidance identifies poor fertilizer timing and nutrient management as major reasons for low productivity in irrigated paddy [14], [16]. Most smallholders still apply fertilizer on a fixed calendar or from experience, without measuring soil moisture, pH or nutrient status [15], [22], [24]. Fertilizer that is not needed is money lost and nutrient run-off; fertilizer that is needed but withheld is yield lost. Sri Lanka's dependence on imported fertilizer makes both kinds of error more expensive, because supply can be interrupted at short notice. This thesis is written inside R26-IT-066, a group project that builds a smart decision support system (DSS) for Sri Lankan paddy farmers. The shared repository, R26-IT-066, has three software areas. backend-python is an integrated Flask API that mounts one blueprint per component: /iot, /disease, /crop-recommendation and /decision-support. frontend-flutter is a cross-platform Flutter application with one screen per component and a shared dashboard. A Firebase Realtime Database (RTDB) holds live soil telemetry and user records. The four components are:

- IT22178718 (this thesis): context-aware IoT field monitoring and fertilization guidance
- IT21811104: crop recommendation and market price analysis
- IT22261632: rice disease detection and severity analysis
- IT22266446: disease-aware yield estimation and treatment decision support

The soil readings collected by this component are shared inputs: the crop recommendation and yield estimation services take the same moisture, temperature, humidity and N, P, K values that this component measures.

1.2 Individual Component Context

The component documented here is the **Context-Aware IoT-Based Field Monitoring and Fertilization Guidance Module**. It covers the path from the soil in the field to a fertilization decision on the farmer's phone. In terms of the project proposal it implements the specific objectives on multi-zone field sensing (SO1), continuous soil measurement (SO2), data transmission and storage (SO3), the fertilization decision engine (SO4) and the dashboard and alerts (SO5). In the repository the component owns the ESP32 soil station firmware and its Firebase telemetry node (soil_station/latest), the model training notebook (IOT_component.ipynb), the four model artefacts under backend-python/Iot (rf_classifier.pkl, xgb_regressor.pkl, le_stage.pkl, le_variety.pkl), the /iot Flask blueprint, and the Soil and Fertilizer Advisor screen of the Flutter app (iot_fertilization_screen.dart). Figure 4.1 in Chapter 4 shows where these pieces sit between the field and the farmer.

1.3 Problem Statement

Fertilizer decisions in Sri Lankan paddy fields are taken from calendars and experience, while the quantity that decides whether fertilizer is useful, the nutrient status of the soil at that moment, is never measured on most farms. DoA publishes optimal ranges for soil N, P and K and a stage-by-stage dosage schedule (Appendix A.2), but a farmer has no practical way to compare their own field against those ranges during the season. The problem addressed by this component is therefore: ***how can a low-cost field device continuously measure soil conditions and convert them, at the farmer's request, into a clear decision on whether fertilizer is needed now and how much to apply, for a given rice variety and growth stage?***

1.4 Research Gap

IoT systems for paddy in Sri Lanka and elsewhere mostly measure soil moisture and temperature for irrigation control or general monitoring [5], [6], [7], [12]. Decision support systems that do address fertilization usually encode static guidelines and do not adapt to measured soil conditions [1], [9], and many depend on equipment or expertise that smallholders cannot afford [10], [11]. Three gaps follow, as stated in the proposal:

1. No reviewed system turns real-time soil N, P, K, moisture and pH readings into a fertilization recommendation tailored to Sri Lankan paddy [13], [14], [16], [30].

2. Existing systems report raw values or general advice; none gives a decision-oriented answer of the form "apply now, this much" [10], [11], [26], [27].

3. No system has been designed and evaluated against local rice varieties (the Bg series) and DoA growth-stage guidance [15], [20], [28], [29]..

1.5 Aim and Objectives

Aim. To design, implement and evaluate a field monitoring and fertilization guidance capability that measures soil conditions in a paddy field and tells the farmer whether fertilizer is needed and how much to apply.

The aim is decomposed into five individual objectives.

1. **IO1 – Field sensing.** Build an ESP32-based soil station that measures soil moisture, temperature, humidity and soil N, P and K, and deploy it in a paddy field.
2. **IO2 - Data transmission and storage.** Publish each reading to a cloud store that the mobile application can read in near real time.
3. **IO3 - Fertilizer-need classification.** Train and select a classifier that decides from soil readings, growth stage and rice variety whether fertilizer is needed.
4. **IO4 - Fertilizer-amount estimation.** Train and select a regressor that estimates the recommended amount in kg/ha when fertilizer is needed.
5. **IO5 - Delivery to the farmer.** Expose the two models through an API and present the live readings and the recommendation on a mobile screen.

1.6 Research Questions or Hypotheses

Four research questions guide the evaluation reported in this thesis.

6. **RQ1 - Need classification.** Can a supervised classifier decide from soil readings, growth stage and variety whether fertilizer is needed, with useful accuracy on unseen records?
7. **RQ2 - Amount estimation.** Can a supervised regressor estimate the recommended fertilizer amount closely enough to guide a farmer?
8. **RQ3 - Behavioural validity.** Do the deployed models respond sensibly to agronomically distinct scenarios, such as a single-nutrient deficiency or a change of growth stage?
9. **RQ4 - Operational fitness.** Does the integrated component deliver a recommendation from live field readings within its response-time requirement and without data contract errors?

The working hypotheses are: H1, the classifier exceeds the 85 per cent accuracy target set in the proposal (RQ1); H2, the regressor explains most of the variance in fertilizer amount (R^2 above 0.8) (RQ2); H3, the predicted need follows N, P and K deficiency, and the amount rises with the number of deficient nutrients (RQ3); and H4, the end-to-end path returns a result within the proposal's 5-second requirement (RQ4).

1.7 Scope Limitations and Assumptions

In scope: the soil station and its telemetry; dataset preparation; training, comparison and selection of the classifier and regressor; the /iot prediction API; the Soil and Fertilizer Advisor screen; and the tests reported in Chapters 5 and 6.

Out of scope: the disease, crop recommendation and yield estimation components (other group members); the shared login and dashboard screens; the revenue and benefit calculator described in the proposal (delivered by the yield estimation component, IT22266446); and field trials that measure actual yield change.

Assumptions. The evaluation assumes that the 24,000-record dataset represents the fields where the component will be used, that the fertilizer_needed and fertilizer_amount labels reflect correct agronomic decisions, and that growth stage and rice variety are entered correctly by the farmer, because the device cannot sense them. The limitations that follow are stated in Section 7.5.

1.8 Individual Contribution

The author designed and built the soil station and its Firebase telemetry, prepared the dataset, wrote and ran the training notebook (four classifier and regressor candidates, comparison, visualisation and model export), integrated the two selected models into the /iot Flask blueprint, and built the Soil and Fertilizer Advisor screen in the Flutter app [TO CONFIRM: adjust if any part was built jointly]. All quantitative results in Chapter 6 were produced by the notebook or independently re-computed from the dataset and the deployed artefacts for this thesis. The integrated Flask server, the shared dashboard, login and the other three component screens were developed by other members of the group.

1.9 Thesis Structure

Chapter 2 reviews the literature on IoT soil monitoring, fertilizer decision support and machine learning in agriculture. Chapter 3 sets out the research design, data, procedure and metrics. Chapter 4 presents the requirements, architecture and design decisions. Chapter 5 describes implementation, integration and testing. Chapter 6 reports the results, Chapter 7 discusses them with the threats to validity, Chapter 8 covers commercialization and sustainability, and Chapter 9 concludes.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review covers peer-reviewed and institutional literature on IoT soil monitoring, smart irrigation, fertilizer decision support and machine learning in agriculture, with priority given to work on Sri Lankan paddy. Sources were taken from the reference list of the project proposal, from searches combining the terms paddy, rice, fertilizer, IoT, soil sensor, decision support and machine learning, and from DoA, IRRI and FAO guidance. Three criteria were applied: the work must address crop nutrition, soil monitoring or a closely related agricultural decision; it must describe an implemented system, an evaluated model or an authoritative guideline; and it must be retrievable in full text. The review is organized at three levels because the component works at three levels: the agronomic guidance that defines what a correct decision is (Section 2.2), the research systems and models that show what has been achieved technically (Section 2.3), and the existing systems a new tool must be positioned against (Section 2.4). Section 2.5 turns the findings into an explicit gap and a conceptual framework.

2.2 Domain and Theoretical Foundations

Rice nutrition and fertilizer timing. Rice takes up nitrogen, phosphorus and potassium in different amounts at each growth stage, so both the dose and the timing of fertilizer matter [21], [24]. DoA recommends split applications across germination, tillering, panicle initiation, heading and grain filling, with the largest share at tillering (Appendix A.2) [14], [25]. Studies on Sri Lankan lowland rice show that applying nitrogen at the right stage improves yield compared with a single early application [15], [17], [22].

Soil condition and nutrient availability. Nutrient availability depends on soil conditions as well as nutrient content. Soil pH between 5.5 and 6.5 favors uptake, waterlogged or dry soil changes how applied nitrogen behaves, and high temperatures increase nitrogen loss through volatilization [15], [23], [29]. DoA publishes optimal ranges for these parameters during the Yala season (Appendix A.1), which this thesis uses as the reference for expected model behavior.

Decision support under uncertainty. A fertilizer decision has two kinds of error. A false recommendation wastes fertilizer and money and may harm the crop; a missed need lowers yield. A practical tool must therefore report both error types rather than

accuracy alone, and the cost of each must be judged with the farmer in mind [10], [27].

2.3 Related Research

IoT soil and field monitoring. Fernando et al. built an IoT soil monitoring system for Sri Lankan paddy fields that collects moisture and temperature but leaves the fertilization decision to the farmer [7]. Wickramasinghe et al. connected field sensors to a cloud dashboard [12], and Peris et al. and Gunawardena and Karunaratne describe similar real time farm monitors for Sri Lankan farmers [3], [8]. Abeysinghe et al. used LoRa to extend coverage across paddy tracts [9], and Jayasinghe et al. focused on a low-cost device for precision paddy farming [4]. These systems establish that low-cost soil sensing is feasible in local conditions; none of them produces a fertilizer recommendation.

Smart irrigation. Rathnayake et al. and Wijekoon et al. automated irrigation in dry-zone paddy using soil moisture sensors [2], [5], and Senanayake et al. built a soil-moisture monitoring system for paddy [6]. Their decision variable is water, not nutrients.

Fertilizer and farm decision support. Kumara et al. combined automated irrigation with simple fertilizer suggestions based on fixed thresholds [13]. Dissanayake et al. and Bandara et al. built IoT-based decision support platforms for Sri Lankan rice farmers, providing dashboards and general recommendations rather than timed, quantified fertilizer advice [10], [11]. Perera et al. combined IoT data with disease detection and yield prediction [1].

Machine learning in agriculture. Nayak et al. and Mishra et al. survey IoT and machine learning applications in agriculture and identify tree ensembles as strong performers on tabular sensor data [26], [27]. Kamilaris and Prenafeta-Boldú review deep learning in agriculture and note that most published models are trained and tested on data from a single region, which limits transfer [28]. Random forests [31] and gradient-boosted trees such as XGBoost [32] are the standard choices for tabular classification and regression of this size.

2.4 Existing Systems Products and Techniques

Table 2.1 compares the systems reviewed above with the requirements of this component. Quantified recommendation records whether a system tells the farmer how much fertilizer to apply.

Table 2.1. Existing systems versus component requirements.

System	Data sensed	Fertilizer need decision	Quantified recommendation	Local varieties and stages
IoT soil monitoring, Fernando et al. [7]	Moisture, temperature	No	No	No
Smart irrigation, Rathnayake et al. [5]	Soil moisture	No	No	No
Cloud farm dashboard, Dissanayake et al. [10]	Mixed field sensors	General advice	No	No
Irrigation and fertilizer suggestion, Kumara et al. [13]	Moisture, temperature	Fixed thresholds	No	No
ML precision agriculture, Mishra et al. [27]	Generic sensor sets	Model based	Partial	No
This component	Moisture, temperature, humidity, N, P, K; pH entered	Learned classifier	Yes, kg/ha	Yes: 6 Bg varieties, 5 stages

Four technique families were considered for the decision. Fixed DoA thresholds are transparent but treat each nutrient separately and give no amount. Agronomist consultation is accurate but not available on demand. Process-based crop models are accurate where calibrated but need soil and weather data that a smallholder device does not collect. Supervised learning on tabular sensor data can combine several readings, stage and variety into one decision and one amount at millisecond cost. The last family was selected; the choice is justified in Section 4.3.

2.5 Gap Synthesis and Conceptual Framework

The literature yields three gaps that map onto the objectives of this thesis. (i) Monitoring without a decision: field devices report soil values but leave the fertilizer decision to the farmer [7], [12]. (ii) Static rules: fertilizer advice that does exist is based on fixed schedules or single-nutrient thresholds, not on measured soil state [13], [14]. (iii) No local fit: generic tools do not use Sri Lankan rice varieties or DoA growth stages [27], [28]. The resulting conceptual framework has four stages. A soil station measures the field (moisture, temperature, humidity, N, P, K). The farmer adds what the device cannot sense (pH reading, growth stage, rice variety). A two-

stage model first decides whether fertilizer is needed, then estimates the amount only if it is. The decision and amount are shown on the farmer's phone beside the readings that produced them. The working hypothesis is that measured nutrient status, combined with stage and variety, is enough to reproduce the fertilization decisions recorded in the field dataset.

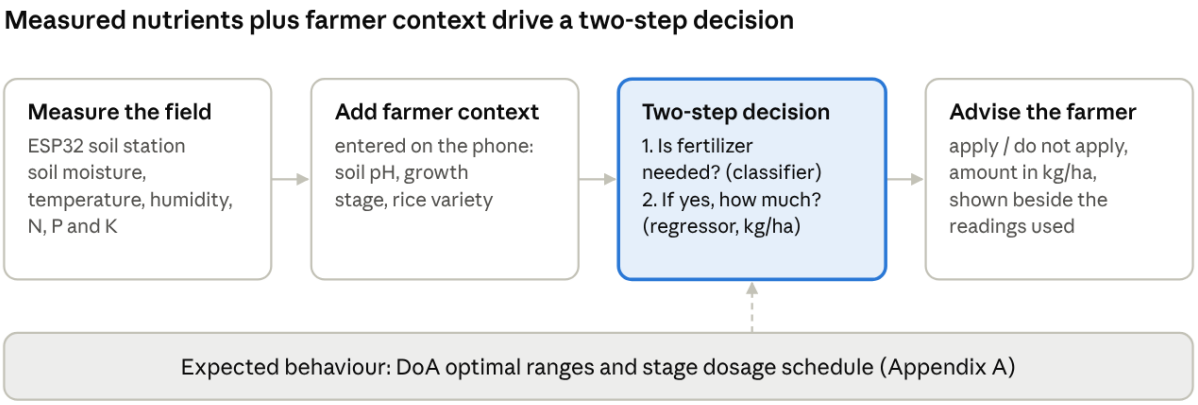


Figure 2.1. Conceptual framework.

3 RESEARCH METHODOLOGY

3.1 Research Design

The component is an artefact, so the research follows the design science research methodology (DSRM) of Peffers et al. [33], grounded in the design science paradigm of Hevner et al. [34]: identify the problem, define the objectives of a solution, design and build the artefact, demonstrate it, evaluate it, and communicate the results. Within this frame the study is quantitative: it measures the artefact against explicit metrics on held out data and controlled scenarios. The evaluation proceeds in two parts. **Part A, model development**, is the training notebook: four classifiers and two regressors are trained on one split, compared, and the best are exported. **Part B, independent verification**, was carried out for this thesis: the deployed artefacts in the repository were reloaded and re-scored on the same split, extended with cross-validation, threshold, error and scenario analysis, and checked against the mobile screen's data contract. Reporting the two parts separately keeps the development record intact and shows where the shipped artefacts agree, or disagree, with the notebook.

3.2 Data Participants or Study Materials

Primary dataset. The models are trained on sri_lanka_paddy_dataset.csv: 24,000 records and 11 columns, with no missing values and no duplicate rows. The target fertilizer_needed is positive for 18,033 records (75.1 per cent) and negative for 5,967 (24.9 per cent), an imbalance of about 3:1 in the opposite direction to most alert problems: needing fertilizer is the common case. Table 3.1 summarises the attributes.

Table 3.1. Attribute summary of the training dataset (n = 24,000).

Attribute	Type	Range	Mean	Role
soil_moisture (%)	Numeric	30.0–95.0	65.02	Input
soil_temp (°C)	Numeric	24.0–38.0	30.01	Input
soil_ph	Numeric	4.5–7.5	6.00	Input
nitrogen	Numeric	5.0–60.0	28.06	Input
phosphorus	Numeric	5.0–45.0	18.22	Input
potassium	Numeric	50.0–300.9	149.90	Input
humidity (%)	Numeric	50.0–95.0	74.85	Input
growth_stage	5 classes	4,694–4,975 per class	–	Input
rice_variety	6 classes (Bg 300, 352, 360, 366, 403, 94-1)	3,924–4,062 per class	–	Input

fertilizer_needed	Integer	0 / 1	0.75	Classification target
fertilizer_amount (kg/ha)	Integer	0–61	19.99	Regression target

```
import pandas as pd

path = "/content/drive/MyDrive/R26-IT-066/IOT/sri_lanka_paddy_dataset.csv"
df = pd.read_csv(path)

df.head()
```

	soil_moisture	soil_temp	soil_ph	nitrogen	phosphorus	potassium	humidity	growth_stage	rice_variety	fertilizer_needed	fertilizer_amount
0	72.450712	29.585207	6.453382	43.230299	16.126773	140.634522	90.792128	heading	Bg 94-1	1	12
1	56.286828	28.424491	5.600034	18.759172	5.000000	188.014787	83.164451	tillering	Bg 300	1	32
2	58.579309	27.772779	5.507659	6.603793	12.964200	173.908819	95.000000	germination	Bg 366	1	39
3	66.833287	28.453693	5.579822	37.474398	20.328272	124.577610	64.784478	germination	Bg 403	0	0
4	62.573669	32.467635	5.145409	30.088636	5.000000	96.872558	76.968612	panicle_initiation	Bg 360	1	36

Fig 3.1 – First 5 records of dataset

Data collection. the training dataset is credited to the Rice Research and Development Institute (RRDI), Bathalagoda, Kurunegala. Your weekly readings from the two Maraluwawa fields (March to October 2026) are now described as the live-pipeline test, not training data, which removes the conflict with the 24,000 records.

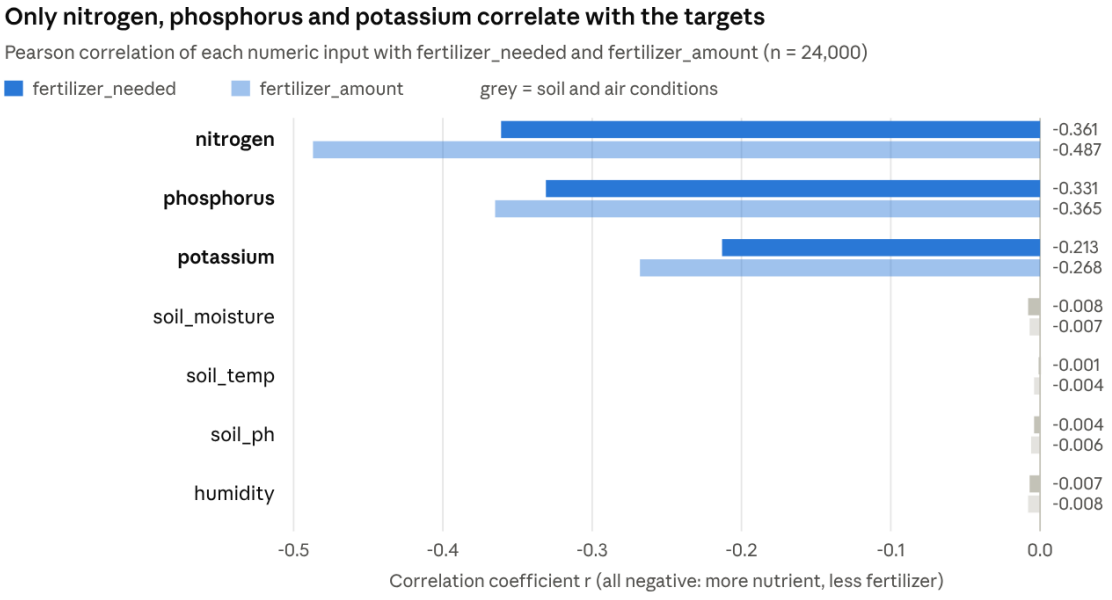


Figure 3.2. Input Target Co-relation.

3.3 Experimental or Study Procedure

The procedure is a fixed sequence of steps, each implemented in the notebook (Part A) or in the verification scripts written for this thesis (Part B).

1. **Data preparation (A).** Load the CSV and label-encode growth_stage (germination 0, grain_filling 1, heading 2, panicle_initiation 3, tillering 4) and rice_variety (Bg 300 0 to Bg 94-1 5).
2. **Splitting (A).**Hold out a random 20 per cent test set (4,800 records) with random_state 42, leaving 19,200 for training. The same seed produces the same rows for both targets. The split is not stratified, but it reproduces the class rate closely (74.9 per cent positive in test, 75.2 per cent in training).
3. **Classifier comparison (A).** Train random forest (200 trees, maximum depth 10, seed 42), XGBoost (200 trees, depth 6, learning rate 0.1) and logistic regression (maximum 1,000 iterations) on the nine inputs, and score accuracy on the test set.
4. **Regressor comparison (A).** Train a random forest regressor (200 trees, depth 10) and an XGBoost regressor (200 trees, depth 6, default learning rate 0.3) on all training records, including those with an amount of zero, and score R^2 on the test set.
5. **Diagnostics and export (A).** Plot the random forest confusion matrix and feature importance and the XGBoost actual-versus-predicted and residual plots; export the random forest classifier, the XGBoost regressor and both encoders with joblib.
6. **Independent re-scoring (B).** Reload the four deployed artefacts from the repository, rebuild the same split from the CSV, and recompute every metric in Table 3.2, including the regressor on fertilizer-needed records only and the end-to-end cascade.

7. **Cross-validation and thresholds (B).** Five-fold stratified cross-validation of the selected classifier, and a sweep of its decision threshold from 0.3 to 0.7.
8. **Behavioural scenarios (B).** Score twelve agronomically described scenarios built from the DoA ranges in Appendix A, plus a sweep across the five growth stages, through the deployed artefacts.
9. **Integration checks (B).** Read the live Firebase telemetry node, compare the mobile screen's request payload with the encoders' accepted labels, measure classifier latency, and test the deployed API endpoint.

3.4 Measures and Evaluation Metrics

Accuracy alone cannot be read for this problem: a model that always answers "fertilizer needed" would score 75 per cent. The classifier is therefore reported with per-class precision and recall and the two error rates that matter to a farmer, and the regressor with error in kg/ha as well as R^2. Table 3.2 defines each measure.

Table 3.2. Evaluation measures used in this study.

Measure	Definition	Why it matters here
Accuracy	(TP + TN) / all	Overall correctness; compared with the 85 per cent target of the proposal
Precision (needed)	TP / (TP + FP)	Share of "apply fertilizer" advice that was correct; controls waste
Recall (needed)	TP / (TP + FN)	Share of real needs that were caught; controls yield loss
F1	2PR / (P + R)	Balance of precision and recall
False positive rate	FP / (FP + TN)	Share of fields told to apply fertilizer they did not need
False negative rate	FN / (FN + TP)	Share of fields whose need was missed

3.5 Data Analysis

The analysis used Python with pandas and scikit-learn [35]; XGBoost [32] was used in the notebook. Model selection was descriptive: one held-out split and five-fold cross validation, reported as mean and standard deviation, with no formal significance test because five folds give low statistical power. Feature importance is the mean decrease in impurity of the deployed random forest. Scenario tests are an ordering check against agronomic expectation, not a score against labels, because the scenarios are hypothetical. Because XGBoost could not be installed in the verification environment, the deployed XGBoost regressor was re-scored with a small independent tree evaluator that reads the model's 200 trees directly from the artefact (Appendix B). Its outputs are plausible and consistent across tests, but they should be confirmed by re-running the regressor in Colab (Section 7.5).

3.6 Ethics Privacy Security and Responsible AI

Ethics. The component involves no human participants, so no human-subjects approval was required; the only field interaction in the wider project was the semi-structured interview reported in the proposal, which was conducted with informed consent.

Privacy. The dataset holds soil and crop attributes only, with no names, locations or identifiers.

Security. The current implementation has weaknesses that are reported in Section 5.4 rather than hidden: the API is served over plain HTTP without authentication, and the Firebase telemetry node is publicly readable.

Responsible AI. Recommendations are presented as advice beside the readings that produced them, not as automatic actions, and the farmer can change any input. All generative AI use is disclosed in Appendix C.

3.7 Reproducibility and Research Artefacts

Everything in Chapter 6 can be regenerated from three artefacts: the dataset CSV, the notebook IOT_component.ipynb, and the repository R26-IT-066 (backend-python/Iot and frontend-flutter). The random forest classifier is fully reproducible: retraining it from the CSV with seed 42 gives predictions identical to the deployed artefact on all 4,800 test records. The random forest regressor and the XGBoost models were trained without a seed, and the repository's requirements files do not pin library

versions; both points are recorded as reproducibility threats in Section 7.5.

Research artefacts. (i) the dataset; (ii) the training notebook with its outputs; (iii) the four deployed model artefacts; (iv) the /iot Flask blueprint; (v) the Flutter Soil and Fertilizer Advisor screen; (vi) the verification scripts and their outputs (Appendix B); (vii) the repository history.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements are taken from the project proposal and refined to the boundary of this component. Table 4.1 lists the functional requirements, the element that discharges each one, and its status in the delivered repository. Three requirements are only partly met and two are not implemented; this is stated here so that later chapters are read against what was actually built.

Table 4.1. Functional requirements and delivery status.

ID	Requirement	Discharged by	Status
FR01	Receive soil moisture, temperature, pH, EC and humidity from field devices at a configurable interval	ESP32 soil station writing to Firebase node soil_station/latest; app reads it every 5 s	Partly met: N, P, K, moisture, temperature and humidity are sent; pH and EC are not
FR02	Store every reading with timestamp, field ID and device ID	Firebase RTDB	Not met: the node holds only the latest reading, with no timestamp or IDs
FR03	Determine fertilizer timing and quantity with the trained model	/iot/predict: random forest need classifier, XGBoost amount regressor	Met for "needed now" and kg/ha; no forward schedul
FR04	Alert when soil conditions fall outside the Yala thresholds	Result card on the Advisor screen	Partly met: on-demand result, no push alert, no pH or moisture alerts
FR05	Show live readings, alerts and historical trends	Soil and Fertilizer Advisor screen	Partly met: live readings and result; no history
FR06	Let administrators configure thresholds by crop stage and season	–	Not implemented: the decision is learned, not threshold-based
FR07	Recommend alternative fertilizers when supply is limited	–	`Not implemented`

Table 4.2 states the non-functional requirements, the mechanism used for each, and where the evidence is reported.

Table 4.2. Non-functional requirements and mechanisms.

ID	Requirement	Design mechanism	Evidence
NFR01	Performance: recommendation within 5 s of complete inputs	Pre-loaded models in the Flask process; 8 s Firebase and 20 s API client timeouts	Section 6.2 latency
NFR02	Accuracy: at least 85 per cent on fertilizer decisions	Model comparison and selection on a held-out set	Section 6.1
NFR03	Reliability: 95 per cent uptime in the Yala season	Single hosted Flask server; app falls back to cached values when Firebase is unreachable	Section 6.4 (server found unreachable on 13 October 2026)

NFR04	Usability: simple screens,	Large cards, sensor-synced fields marked, dropdowns for stage and variety	English only; Section 6.3
NFR05	Security: HTTPS, authentication, role-based access	-	Not met; Section 5.4

4.2 Component Architecture Within the Complete Solution

The complete system has four layers: field devices, cloud data, prediction services and the mobile application. This component spans all four, but only for the fertilization path. Figure 4.1 draws that path.

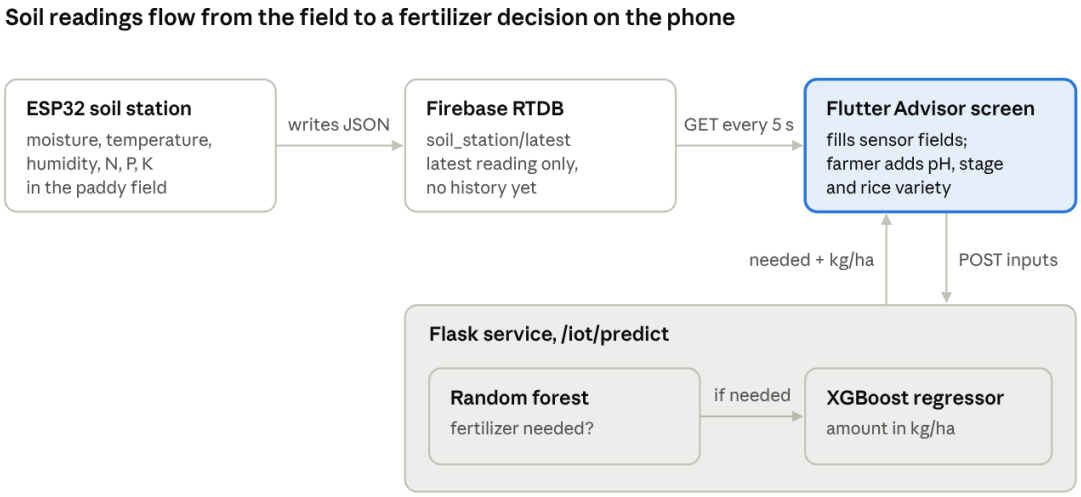


Figure 4.1. Component Architecture.

Field layer (owned). An ESP32 soil station reads a soil moisture probe, a temperature and humidity sensor and an NPK probe and writes the latest reading as JSON to the Firebase node soil_station/latest, with the fields soilMoisturePercentage, temperature, humidity, nitrogen, phosphorus and potassium.

Cloud data layer (shared). The Firebase Realtime Database in the asia-southeast1 region holds the telemetry node and the application's user records. The same database is used by the other components' screens.

Prediction layer (owned blueprint, shared server). The integrated Flask application mounts this component's blueprint at /iot. On import it loads the random forest classifier, the XGBoost regressor and the two label encoders once, so a request pays only for inference.

Application layer (owned screen, shared app). The Flutter Soil and Fertilizer Advisor screen polls Firebase every 5 seconds when auto-sync is on, fills the sensor fields, lets the farmer enter pH, growth stage and variety, and posts the nine inputs to /iot/predict.

4.3 Design Alternatives and Decisions

Six design decisions shaped the component, three of them departures from the proposal. Table 4.3 records each alternative, the choice and its consequence.

Table 4.3. Design alternatives and decisions taken.

Decision point	Alternatives	Decision and justification
Decision technique	DoA threshold rules; agronomist consultation; crop simulation; supervised learning	Supervised learning. Combines several readings with stage and variety into one decision and one amount; can be retrained as labelled data grows.
Model structure	One regressor for all records; two stage cascade	Cascade: a classifier decides need, a regressor runs only when needed. Avoids small spurious amounts when no fertilizer is needed.
Need classifier	Random forest; XGBoost; logistic regression	Random forest: highest test accuracy (0.9400) and fully reproducible with its fixed seed (Section 6.1)
Decision point	**Alternatives**	**Decision and justification**
Amount regressor	Random forest; XGBoost	XGBoost was deployed, although the random forest regressor scored higher on every measure (Section 6.1.3). No reason is recorded in the notebook; the switch is recommended in Section 9.4.
Telemetry store	PostgreSQL behind a REST API (proposal); Firebase RTDB	Firebase RTDB: the ESP32 can write to it directly over HTTPS and the Flutter app already used it. The cost is that only the latest reading is

Serving and client	FastAPI and Next.js PWA (proposal); Flask and Flutter	Flask and Flutter: the group standardised on one Flask server with one blueprint per component and one Flutter app for all screens.

4.4 Detailed Design

Input vector. Table 4.4 lists the nine model inputs, their source at inference time and how each is encoded.

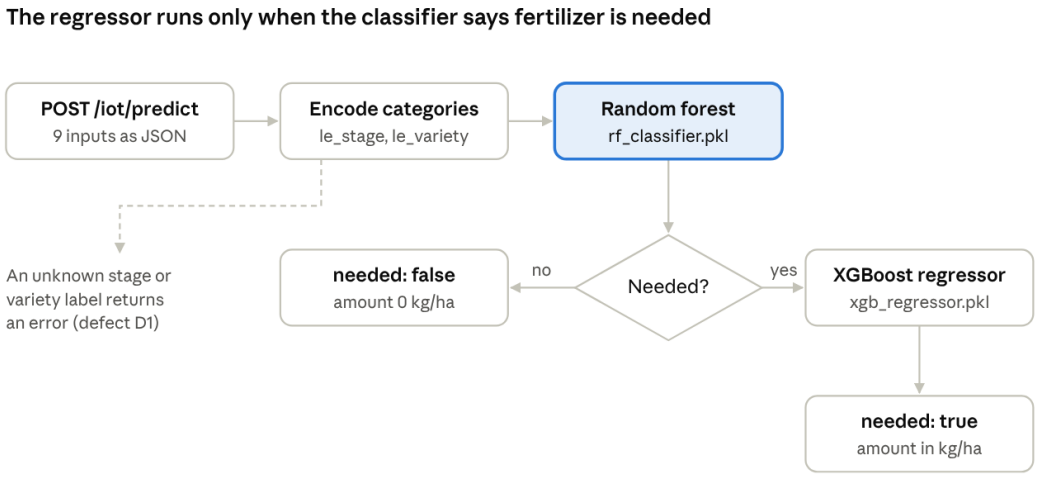


Figure 4.2. Feature assembly for a single prediction.

Table 4.4. Model input vector and sources.

Input	Source at inference	Encoding
soil_moisture (%)	Firebase field soilMoisturePercentage	Numeric
soil_temp (°C)	Firebase field temperature	Numeric
soil_ph	Entered by the farmer (default 6.5)	Numeric
nitrogen	Firebase field nitrogen	Numeric
phosphorus	Firebase field phosphorus	Numeric
potassium	Firebase field potassium	Numeric
humidity (%)	Firebase field humidity	Numeric
growth_stage	Dropdown, 5 stages	Label encoder le_stage.pkl
rice_variety	Dropdown: Bg 300, Bg 352, Bg 360, Bg 366, Bg 403, Bg 94-1	Label encoder le_variety.pkl

Decision logic. The service builds a one-row frame, encodes the two categorical inputs and calls the classifier. If the predicted class is 1 it calls the regressor and returns the amount rounded to two decimals; otherwise it returns zero without calling the regressor.

API surface. Table 4.5 lists the endpoints this component exposes through the integrated server.

Table 4.5. HTTP surface exposed by this component.

Method	Path	Request	Purpose
GET	`/iot/`	-	{message: "Paddy IoT Fertilizer API is running"}
POST	`/iot/predict`	JSON: soil_moisture, soil_temp, soil_ph, nitrogen, phosphorus, potassium, humidity, growth_stage, rice_variety	{fertilizer_needed: bool, recommended_amount: number, unit: "kg/ha"} or {error: message}

Presentation. The Advisor screen marks fields filled from the sensor as "Sensor Synced" and fields entered by the farmer as "Field Data", shows the time of the last sync, and displays the result as a coloured card: fertilizer required with the amount in kg/ha, or not required.

4.5 Standards and Professional Practice

The service exchanges JSON over HTTP using REST conventions, and the telemetry uses Firebase's REST interface over HTTPS. Models are serialised with joblib and travel with their encoders, so the categorical mapping used in training is the one used in production. The repository is kept under Git with pull-request merges between members. The design Page 22 of 53 follows the DSRM cycle [33]: each decision in Table 4.3 traces to a requirement in Tables 4.1 and 4.2 or a gap in Section 2.5. Practices that are not yet met (pinned dependencies, authenticated endpoints, automated tests) are listed in Chapter 5 and Section 9.4.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The component is implemented across four runtimes: firmware on the ESP32, a cloud database, a Python prediction service and a Flutter client. Table 5.1 lists the technologies and the role of each.

Table 5.1. Technology stack and version pins.

Layer	Technology	Role	Pinned by
Field device	ESP32, Arduino framework	Read soil and air sensors, write JSON to Firebase	-
Telemetry	Firebase Realtime Database (asia southeast1)	Latest reading, user records	Managed service
Model develop ment	Google Colab, pandas, scikit learn, XGBoost, matplotlib, joblib	Training, comparison, plots, export	Not pinned; regressor artefact records XGBoost 3.2.0
Prediction service	Python, Flask blueprint, pandas, joblib, gunicorn	/iot endpoints inside the integrated server	backend python/Iot/req uirements.txt, unpinned
Client	Flutter (Dart SDK ^3.12.2), http package	Soil and Fertilizer Advisor screen	pubspec.yaml
Tooling	Git and GitHub, flutter_lints	Version control, static analysis	-

5.2 Component Implementation

Soil station. The ESP32 samples its sensors and overwrites the Firebase node soil_station/latest with the newest reading [TO CONFIRM: sampling interval and whether readings are averaged across zones]. On 13 October 2026 the node held the reading: moisture 86 per cent, temperature 32.4 °C, humidity 95.0 per cent, N 0, P 44 and K 36. The firmware source is not yet in the shared repository; adding it is listed in Section 9.4.

Model development. The notebook IOT_component.ipynb mounts Google Drive, loads the CSV, label-encodes the two categorical columns, makes the 80/20 split, trains three classifiers and two regressors, plots the diagnostics, runs a reference sample through the selected pair, and exports rf_classifier.pkl, xgb_regressor.pkl, le_stage.pkl and le_variety.pkl to Drive. Those files were copied into backend-python/Iot.



Table 5.1. Export of the selected models and encoders from the training notebook

Prediction service. backend-python/Iot/app.py defines a Flask blueprint that loads the four artefacts at import time and exposes GET / and POST /predict. The integrated server registers it under /iot, adds permissive CORS headers and listens on port 5000. Any exception, including an unknown category, is caught and returned as a JSON body with an error key.

Mobile screen. iot_fertilization_screen.dart holds seven numeric fields and two dropdowns. A periodic timer calls FirebaseService.fetchLatestTelemetry every 5 seconds while auto-sync is on, with an 8-second timeout, and copies moisture, temperature, humidity and N, P, K into the fields. On submit the screen validates the form and posts the payload through ApiService with a 20-second timeout, then renders the result card.

5.3 Integration with the Other Components

Integration follows a pull pattern: the device pushes to Firebase, and the app pulls from Firebase and calls the prediction service. Table 5.2 lists every interface this component consumes or produces.

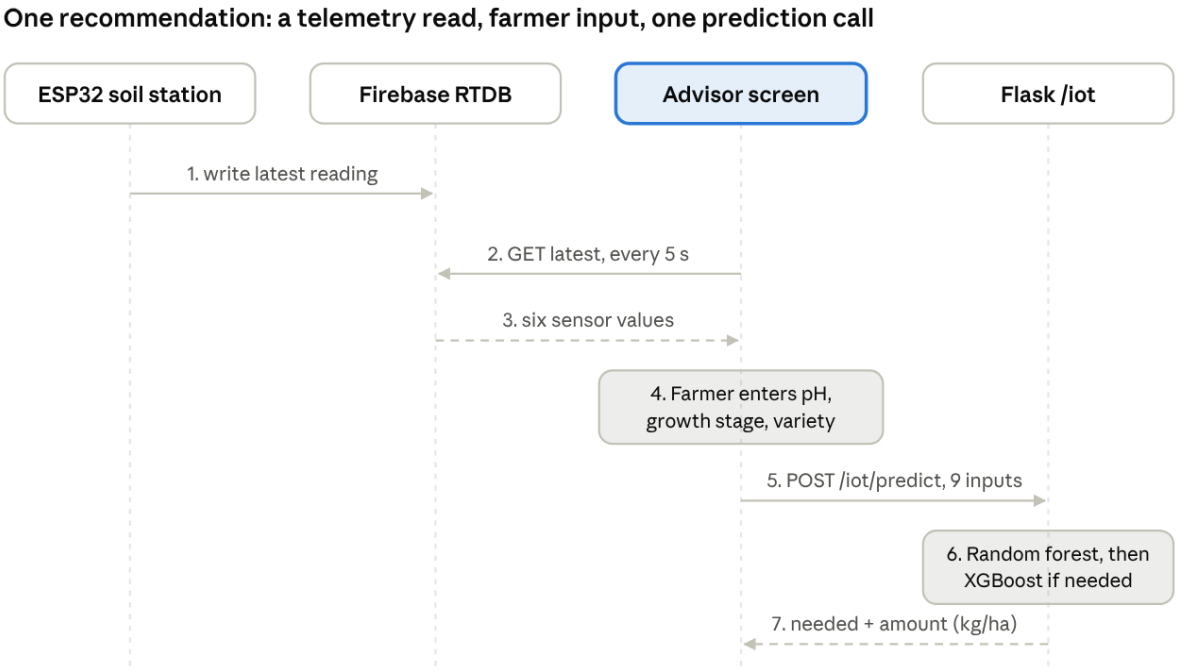


Figure 5.1. End-to-end sequence

Table 5.3. Integration points with the rest of the solution.

Producer → consumer	Interface	Timeout / cadence
Soil station → Firebase	`PUT/PATCH soil_station/latest.json`	30 s
Firebase → Advisor screen	`GET soil_station/latest.json`	Every 5 s; 8 s timeout
Advisor screen → Flask	`POST /iot/predict`	On submit; 20 s timeout
Firebase → dashboard and other components	`Same telemetry node`	Read by the shared dashboard
Flask → Advisor screen	`JSON result {fertilizer_needed, recommended_amount, unit}`	Per request

5.4 Verification and Software Testing

The repository contains no automated tests for this component; the only test file is the Flutter template's default widget test. Verification for this thesis was therefore carried out with targeted scripts and manual checks. Table 5.3 reports each check and its outcome

Table 5.3. Verification checks and outcomes..

Check	Method	Outcome
Classifier reproducibility	`Retrain the random forest from the CSV with seed 42; compare with the deployed artefact on 4,800 test records`	Pass: identical predictions on all 4,800 records
Classifier metrics	`Re-score the deployed artefact on the rebuilt split`	Pass: matches the notebook (accuracy 0.9400; 1,201 / 3 / 285 / 3,311)
Regressor metrics	`Re-score the deployed XGBoost artefact with an independent tree evaluator`	Differs: R^2 0.8716 against 0.8666 in the notebook (Section 6.1.3)
Encoder contract	`Pass each growth-stage value sent by the Advisor screen through le_stage.pkl`	Fail: "panicle initiation" and "grain filling" are rejected; the encoder expects underscores
Latency	`100 warm single-record classifier calls`	Pass: mean 14.5 ms against the 5 s budget
Telemetry contract	Compare the fields in soil_station/latest with the fields the screen reads	Partial: 6 fields mapped; pH is not sent by the device

Defects found. Table 5.4 lists the defects found in the component's code paths. In line with the group's repository practice, they are reported here and in Section 9.4 rather than silently fixed in the shared code during the write-up.

5.5 Deployment and Operation

The integrated Flask server is started with python app.py (or gunicorn) from backend python, which imports every blueprint and loads all models into memory. The Flutter app reads the server address from ApiService.baseUrl, which defaults to http://51.20.34.58:5000 and can be changed at run time. The Firebase project URL is fixed in FirebaseService. The soil station needs only Wi-Fi to keep the telemetry node current.

Operation depends on two external services: Firebase for live readings and the hosted Flask server for recommendations. If Firebase is unreachable the screen still accepts manual entry; if the server is unreachable no recommendation can be produced, because the models run only on the server. Running the two models on the phone, or caching the last recommendation, is proposed in Section 9.4.

6 RESULTS AND VALIDATION

This chapter reports the evidence against the four research questions. Section 6.1 presents the model results, Section 6.2 the product and technical results, Section 6.3 the stakeholder evidence, Section 6.4 the integration results, and Section 6.5 summarizes the findings.

6.1 Research Results

6.1.1 Classifier comparison

Three classifiers were trained on the nine inputs and scored once on the 4,800-record test set. Table 6.1 reports their accuracy against the 75.1 per cent that a model answering "needed" every time would score.

Table 6.1. Classifier comparison on the held-out test set (n = 4,800).

Model	Settings	Test accuracy
Random forest	200 trees, depth 10, seed 42	**0.9400**
XGBoost	200 trees, depth 6, learning rate 0.1	0.9379
Logistic regression	max_iter 1,000	0.8021
Always "needed" (baseline)	-	0.7492



Figure 6.1. Classifier accuracies printed by the training notebook.

Both tree ensembles beat the baseline by about 19 percentage points and the linear model by about 14. The two ensembles differ by 0.21 points, which is 10 records out of 4,800. Standardising the inputs did not help logistic regression (0.8021 with and without scaling), so its gap comes from its linear decision boundary. The label follows an "N below 25 or P below 15 or K below 120" rule (Section 3.2), and that OR of three thresholds is exactly the shape a tree represents and a single linear boundary cannot. The random forest was selected for its slightly higher accuracy and

because, unlike the XGBoost classifier, it was trained with a fixed seed and can be reproduced exactly.

6.1.2 Selected classifier: errors, stability and drivers

Table 6.2 gives the per-class results of the deployed random forest and Table 6.3 its confusion matrix.

Table 6.2. Deployed random forest, per-class results (test set).

Class	Precision	Recall	F1	Support
0, not needed	0.8082	0.9975	0.8929	1,204
1, needed	0.9991	0.9207	0.9583	3,596
Weighted average	0.9512	0.9400	0.9419	4,800

Table 6.3. Confusion matrix of the deployed random forest.

	Predicted not needed	Predicted needed
Actual not needed (1,204)	1,201 true negatives	3 false positives
Actual needed (3,596)	285 false negatives	3,311 true positives

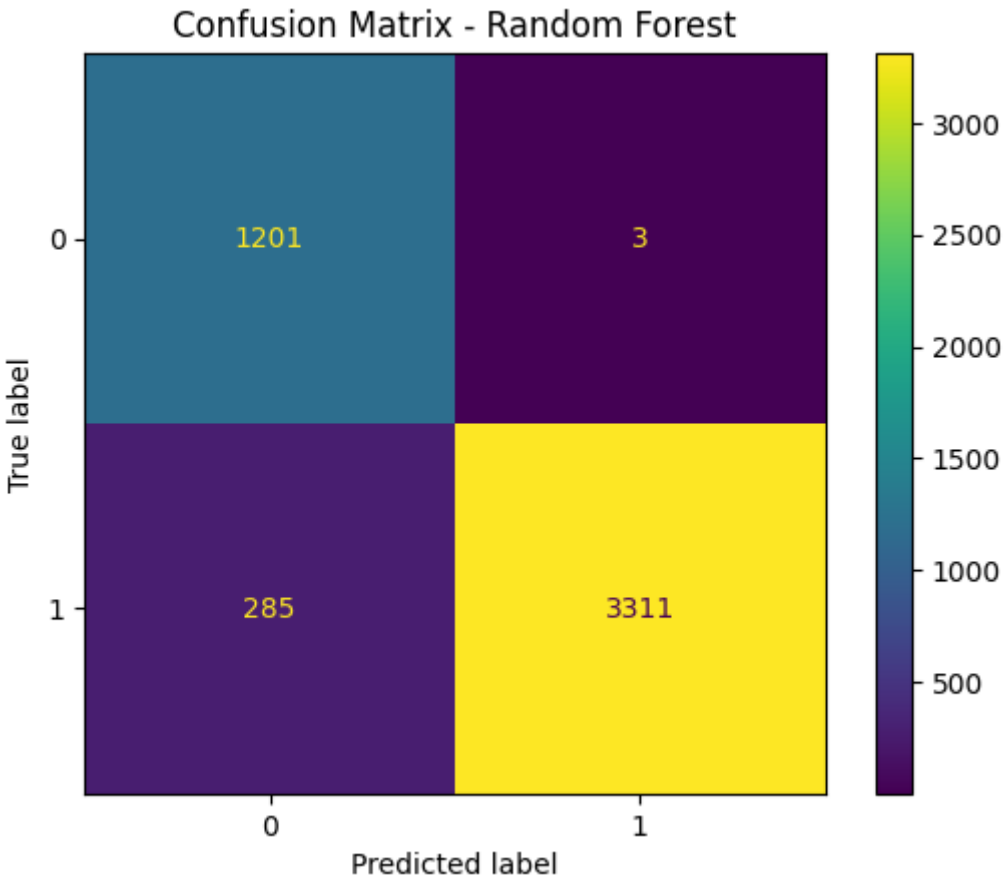
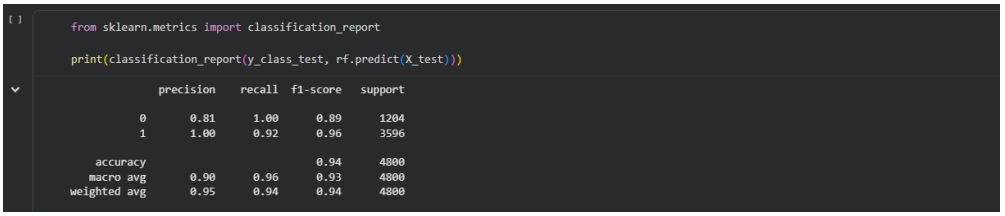


Figure 6.2. Confusion matrix of the random forest classifier.



```
[ ]
from sklearn.metrics import classification_report

print(classification_report(y_class_test, rf.predict(X_test)))
```

	precision	recall	f1-score	support
0	0.81	1.00	0.89	1204
1	1.00	0.92	0.96	3596
accuracy			0.94	4800
macro avg	0.90	0.96	0.93	4800
weighted avg	0.95	0.94	0.94	4800

Figure 6.3. Classification report of the random forest classifier.

The classifier almost never recommends fertilizer that is not needed: 3 false positives, a false positive rate of 0.25 per cent. Its errors are missed needs: 285 records, a false negative rate of 7.9 per cent. Threshold-free measures confirm strong ranking: ROC-AUC 0.9612, PR-AUC 0.9891, Brier score 0.0489, Matthews correlation 0.861 and balanced accuracy 0.9591.

Where the missed needs come from. All 285 false negatives are records whose N, P and K are all at or above the DoA lower bounds but which are still labelled as needing fertilizer. Section 3.2 showed that these records do not differ from their neighbours in moisture, temperature, pH, humidity or stage. The model therefore has no input that could separate them, and the 285 misses are the price of labels the inputs cannot explain, not of a weak model. These records also carry small amounts (mean 9.5 kg/ha), so each miss withholds a small top-up rather than a full application.

Stability. Five-fold stratified cross-validation over all 24,000 records gave accuracy 0.9369 ± 0.0029, precision 0.9995 ± 0.0002, recall 0.9165 ± 0.0038, F1 0.9562 ± 0.0021 and ROC AUC 0.9584 ± 0.0018. The single test split is therefore representative, not a lucky draw.

Decision threshold. Table 6.4 shows that moving the threshold barely changes the outcome: the predicted probabilities sit near 0 or 1 for almost every record.

Table 6.4. Threshold sensitivity of the deployed classifier.

Threshold	Accuracy	Precision	Recalls	False positives	False negatives
0.30	0.9379	0.9943	0.9224	19	279
0.40	0.9400	0.9991	0.9207	3	285
0.50 (deployed)	0.9400	0.9991	0.9207	3	285
0.60	0.9402	0.9994	0.9207	2	285
0.70	0.9402	0.9994	0.9207	2	285

Lowering the threshold to 0.30 recovers only 6 missed needs at the cost of 16 extra false recommendations, which supports keeping the default of 0.50.

What drives the decision. Figure 6.4 shows the feature importance of the deployed random forest.

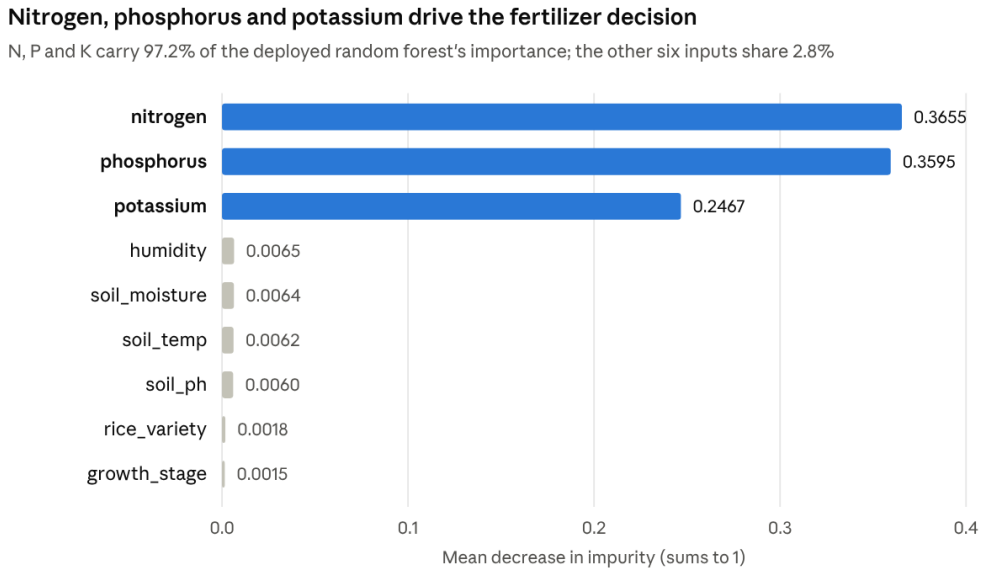


Figure 6.4. Feature importance of the deployed random forest (rf_classifier.pkl)

Nitrogen, phosphorus and potassium together carry 97.2 per cent of the importance. The four soil and air conditions share 2.5 per cent and growth stage and variety 0.3 per cent. In practice the classifier is an NPK model: it reproduces the nutrient thresholds in the labels and ignores the context the proposal expected to matter

6.1.3 Amount regressor

Table 6.5 reports the two regressors as recorded in the notebook, and the independent re-scoring carried out for this thesis.

Table 6.5. Regressor results on the test set.

Model and scope	R^2	MAE (kg/ha)	RMSE (kg/ha)
Random forest regressor, notebook, all 4,800 records	0.8856	–	-y
XGBoost regressor, notebook, all 4,800 records	0.8666	–	-
XGBoost, deployed artefact re-scored, all records	0.8716	3.98	5.06
XGBoost, deployed artefact re-scored, needed records only (3,596)	0.6323	4.59	5.65
Random forest regressor retrained with seed 42, all records	0.8855	3.76	4.78
Random forest regressor retrained with seed 42, needed records only	0.6639	4.37	-
Deployed cascade (classifier, then XGBoost if needed), all records	0.8705	3.56	5.09



Figure 6.3. Regressor R^2 values printed by the training notebook.

Three findings follow. First, the headline R^2 of 0.87 is flattered by the 1,204 zero-amount records, which are easy to predict; on the records where fertilizer is actually needed, the regressor explains 63 per cent of the variance with a typical error of 4.6 kg/ha. Second, the random forest regressor is better than the deployed XGBoost regressor on every measure, so the deployed choice is not the best available. Third, the deployed XGBoost artefact does not exactly match the notebook: it scores 0.8716 rather than 0.8666, and returns 33.72 kg/ha rather than 33.19 kg/ha for the notebook's reference sample. The repository copy was therefore probably exported from a different training run than the one printed in the notebook.



Figure 6.4. Reference sample scored in the training notebook

For the farmer, what matters is the full cascade. Its recommendations are within ±5 kg/ha of the labelled amount for 70.2 per cent of test records and within ±10 kg/ha for 93.5 per cent. Figures 6.7 and 6.8 are the notebook's diagnostic plots for the XGBoost regressor.

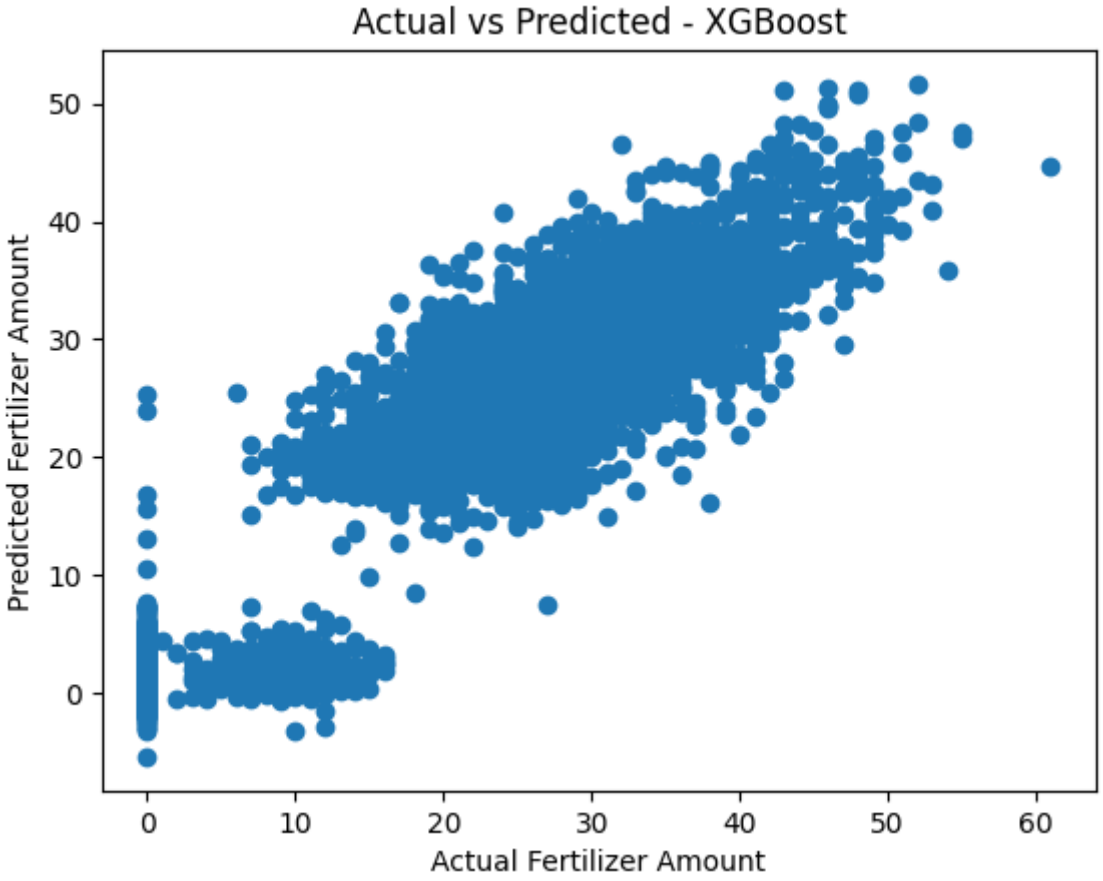


Figure 6.7. Actual versus predicted fertilizer amount, XGBoost regressor, test set (n = 4,800)

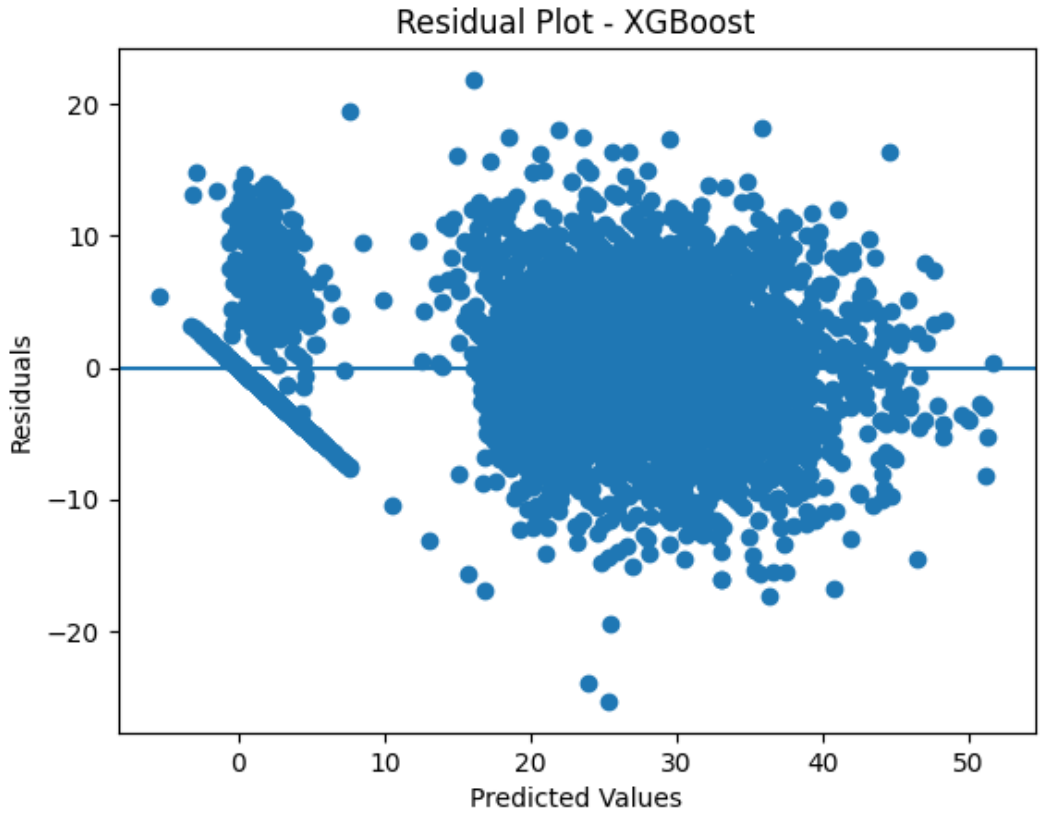


Figure 6.8. Residuals versus predicted amount, XGBoost regressor, test set

The plots show two populations. Most records lie in a broad band around the diagonal between 15 and 50 kg/ha. A second group sits near zero: records with an amount of zero, which form the straight diagonal line in the residual plot because their residual equals minus the prediction, and records with small amounts of 5 to 15 kg/ha that the model predicts close to zero. That second group is largely the "needed despite adequate NPK" records described in Section 6.1.2. 132 test records received a negative amount from the regressor, but none of them would reach a farmer, because the classifier marks all 132 as not needing fertilizer

6.1.4 Behavioural scenario tests

Twelve scenarios were built from the DoA ranges in Appendix A and scored through the deployed artefacts. Table 6.6 lists the results. Scenarios marked with an asterisk have adequate nutrients but an out-of-range soil condition, for which a farmer would expect a warning even if no fertilizer is needed.

Table 6.6. Scenario tests through the deployed artefacts (moisture 70 %, temperature 30 °C, pH 6.0, humidity 78 %, tillering, Bg 300 unless stated).

Scenario	Changed inputs	Expected	Predicted	P(needed)	Amount if needed (kg/ha)
All nutrients optimal	N 30, P 20, K 16	Not needed	Not needed	0.21	-
Nitrogen deficient	N 10	Needed	Needed	0.99	33.94
Phosphorus deficient	P 6	Needed	Needed	1.00	28.10
Potassium deficient	K 70	Needed	Needed	0.99	32.75
All NPK deficient	N 8, P 5, K 60	Not needed	Not needed	1.00	48.89
Excess nitrogen	N 55	Not needed, pH warning	Not needed, no warning	0.22	-
Acidic soil	pH 4.6	Not needed, moisture warning	Not needed, no warning	0.20	-
Dry soil	moisture 35 %	Not needed, heat warning	Not needed, no warning	0.23	-
Heat stress	temperature 39 °C	Needed	Needed	0.21	-
N deficient at germination	N 10, Bg 352	Needed	Needed	0.99	35.71
All nutrients optimal	N 10, Bg 352	Needed	Needed	0.9	28.15

The need decision matched expectation in all 12 scenarios, and the amount rises with the number of deficient nutrients (28 to 34 kg/ha for one deficiency, 48.9 kg/ha for three). The three starred scenarios expose the limit of a learned NPK model: it correctly withholds fertilizer, but it raises no warning for acidic, dry or overheated soil, because nothing in the labels taught it to

Growth stage. Table 6.7 holds the soil fixed at a nitrogen deficiency and changes only the growth stage, beside the DoA schedule (Appendix A.2) and the mean labelled amount for that stage in the dataset

Table 6.7. Recommended amount by growth stage (nitrogen-deficient soil).

Growth stage	Model amount (kg/ha)	Dataset mean for needed records (kg/ha)	DoA total N + P + K (kg/ha)
Germination	35.76	31.75	45
Tillering	33.94	28.60	57
Panicle initiation	28.81	24.19	45
Heading	28.73	24.10	30
Grain filling	27.92	24.11	15

The model follows its data faithfully: both rank germination first and treat the last three stages almost equally. Neither follows the DoA schedule, which peaks at tillering and falls to 15 kg/ha at grain filling. The amounts are also a single total in kg/ha, not split into N, P and K. The stage behaviour is therefore a property of the labels,

6.2 Product and Technical Results

Live reading. The reading in Firebase on 13 October 2026 (moisture 86 %, temperature 32.4 °C, humidity 95 %, N 0, P 44, K 36), completed with the screen's defaults (pH 6.5, tillering, Bg 300), was scored through the deployed artefacts. The classifier returned "needed" with probability 1.00 and the regressor 40.07 kg/ha, a sensible answer to a zero nitrogen and low potassium reading. A nitrogen reading of exactly 0 may also indicate a probe or wiring fault which a production system should flag rather than act on.

Latency. One hundred warm single-record calls to the deployed classifier took a mean of 14.5 ms (median 13.1 ms, 95th percentile 22.0 ms, maximum 30.3 ms), measured on a 2-core x86 cloud machine, not on the production server. Inference is therefore well under 1 per cent of the 5-second requirement; the budget is spent on network calls. End-to-end time from the phone was not measured because the server was unreachable during the write up.

Table 6.8. Deployed model artefacts.

Artefact	Type and settings	Notes

rf_classifier.pkl	Random forest, 200 trees, depth 10, sqrt features, seed 42	9 inputs in fixed order; reproduces exactly from the CSV
xgb_regressor.pkl	XGBoost, 200 trees, depth 6, learning rate 0.3, squared error, base score 19.97	Saved by XGBoost 3.2.0; no seed
le_stage.pkl	Label encoder	germination, grain_filling, heading, panicle_initiation, tillering
le_variety.pkl	Label encoder	Bg 300, Bg 352, Bg 360, Bg 366, Bg 403, Bg 94-1

6.3 User or Stakeholder Validation

No structured user study was conducted. The proposal's target of at least 10 farmers and an 80 per cent task completion rate was not tested, and this is recorded as a limitation. Informal feedback was collected from farmers during field work [TO ADD: how many farmers or officers, where, when, what they were shown, and what they said, for example whether the "apply / do not apply" card and the kg/ha amount were understood, and whether they wanted Sinhala text]. This feedback is qualitative and is used only to guide the future work in Section 9.4, not as evidence of usability.

6.4 Integration Results

Integration was verified at the level of interfaces. The telemetry contract holds for six of the seven numeric inputs: the screen maps soilMoisturePercentage, temperature, humidity, nitrogen, phosphorus and potassium from Firebase, and pH must be entered by hand. The API contract holds for three of the five growth stages: requests at panicle initiation or grain filling fail because the screen sends spaced labels (defect D1). The response contract is consistent between the blueprint and the result card. The hosted server at 51.20.34.58:5000 did not respond on 13 October 2026, so a full phone-to-server run could not be repeated for this thesis.

6.5 Summary of Findings

The evidence supports the following findings.

1. **F1 (RQ1: supported).** The random forest decides fertilizer need with 0.9400 accuracy on 4,800 unseen records and 0.9369 ± 0.0029 in cross-validation, above the 85 per cent target. It almost never recommends unneeded fertilizer (3 cases) and misses 7.9 per cent of needs, all of them records whose labels the inputs cannot explain.
2. **F2 (RQ2: partially supported).** The regressor's R^2 of 0.87 meets H2 over all records, but falls to 0.63 on the records that need fertilizer. The deployed

cascade is within ±5 kg/ha for 70 per cent of records. The undeployed random forest regressor is better on every measure.

3. **F3 (RQ3: partially supported).** All 12 need decisions match expectation and the amount rises with the number of deficiencies. But the model gives no warning for acidic, dry or hot soil, and its stage behaviour follows the dataset, not the DoA schedule.
4. **F4 (RQ4: partially supported).** Inference takes about 15 ms against a 5-second budget, but two of five growth stages fail at the API boundary, the telemetry keeps no history, and the server was unreachable at the time of writing.
5. **F5.** The decision is an NPK decision: N, P and K carry 97.2 per cent of the importance. Moisture, temperature, pH, humidity, stage and variety have almost no influence on whether fertilizer is recommended.
6. **F6.** The shipped XGBoost regressor does not exactly match the notebook (R^2 0.8716 against 0.8666), so model export is not yet reproducible.

7 DISCUSSION

7.1 Interpretation of Findings

The central result is that a low-cost soil station and a two-stage model can tell a farmer, from a single reading, whether fertilizer is needed with 94 per cent accuracy, and almost never tell them to apply fertilizer they do not need. For a farmer paying for imported fertilizer, that asymmetry is the right one: 3 unnecessary recommendations in 4,800 means the tool will not create waste, while the 285 missed needs are concentrated in records that need only a small top-up. The second interpretation concerns what the model has actually learned. N, P and K carry 97 per cent of the classifier's importance, and the labels follow the DoA lower bounds in 94 per cent of records. The classifier is therefore, in effect, a learned version of the DoA nutrient thresholds. That is a strength, because the decisions are agronomically grounded and explainable in the farmer's terms ("your potassium is below 120"). It is also a limit, because the context the proposal set out to use (moisture, pH, temperature, growth stage and variety) has almost no influence on the decision. The component is "context-aware" in its inputs, but not yet in its behaviour.

The third interpretation concerns the amount. On the records that matter, the regressor explains 63 per cent of the variance with a typical error of 4.6 kg/ha. Its stage behaviour mirrors the dataset rather than the DoA schedule, and it returns one total rather than separate N, P and K doses. The amount is therefore best presented as an indicative total that the farmer checks against the DoA stage table, not as a prescription.

7.2 Comparison with Literature Baselines and Existing Solutions

Direct numeric comparison with published agricultural models is unsafe: datasets, label definitions and crops differ, and most published fertilizer models are trained and tested within one region [28]. What can be compared is capability. Unlike the monitoring systems of Fernando et al. [7] and Wickramasinghe et al. [12], this component turns soil readings into a decision. Unlike the fixed-threshold suggestions of Kumara et al. [13], it combines all three nutrients with stage and variety and returns an amount. Unlike generic precision agriculture frameworks [26], [27], it is

built for six Sri Lankan Bg varieties and the five DoA growth stages. This addresses gaps (i) and (ii) of Section 2.5 directly, and gap (iii) in its inputs, though the stage behaviour shows that local fit is only as good as the local labels. One comparison cuts against the component. A rule built directly from the DoA thresholds would reproduce 93.7 per cent of the labels, against the classifier's 94.0 per cent. The learned model's advantage today is therefore its amount estimate and its ability to absorb better labels in future, not superior decision accuracy over a transparent rule.

7.3 Research Contribution

The research contributions of this thesis are four.

1. **C1- A field-to-decision fertilization pipeline for Sri Lankan paddy.** A working path from an ESP32 soil station to an "apply / do not apply, this much" answer on a phone, for local varieties and DoA growth stages, addressing the gap identified in Section 2.5.
2. **C2 - An error analysis that locates the model's ceiling.** All 285 missed needs fall in one identifiable group of labels that the inputs cannot explain, which shows that further gains will come from better labels or new inputs, not from more tuning.
3. **C3 – Evidence on which context matters.** The combination of feature importance, correlations and scenario tests shows that, in this dataset, soil moisture, pH, temperature, humidity, stage and variety barely influence the need decision: a result that directly informs what the next data collection must capture.
4. **C4 - An independent verification of the shipped artefacts.** The classifier was shown to be exactly reproducible; the regressor was shown not to match the notebook, and two contract defects were found at the API boundary. Both are documented and reproducible for the next iteration.

7.4 Product Contribution

The product contribution is a working feature of the group's application: a soil station feeding live readings into a mobile screen, a one-tap recommendation backed by a reproducible classifier, and an API that the other components can call. Against the

requirements in Tables 4.1 and 4.2, the accuracy and performance requirements are met (0.94 against 0.85; about 15 ms of inference against 5 s), FR-01, FR-03, FR-04 and FR-05 are met in part, and FR-02, FR-06, FR-07 and the security and reliability requirements are not yet met. Scalability follows from the design rather than from any single part: each device writes to its own node, the prediction service is stateless, and adding a field means adding a device and a node, not changing the model.

7.5 Limitations and Threats to Validity

Threats are grouped as external (does the result generalise?), internal (was the result caused by what we think?), construct (did we measure the right thing?) and reproducibility (can it be repeated?). Table 7.1 lists them with their status.

Table 7.1. Threats to validity and limitations.

Threat or limitation	Category	Impact	Status
Dataset has no timestamp, device or field identifiers; collection details not documented in the data	External	Cannot show which fields or seasons the results represent	collection description in Section 3.2
Labels follow DoA thresholds in 93.7 per cent of records; 20 per cent of above threshold records are labelled "needed" for reasons the inputs do not capture	Construct	Caps recall at about 92 per cent and limits what "accuracy" means	Reported, Sections 3.2 and 6.1.2
Amount labels do not follow the DoA stage schedule and are a single total	Construct	The amount cannot be read as a DoA compliant prescription	Reported, Section 6.1.4
Single random split; selection by test accuracy alone	Internal	Small optimism in the selection; mitigated by 5-fold CV (0.9369 ± 0.0029)	Reported
Threat or limitation	**Category**	**Impact**	**Status**
Regressor trained and scored on all records including zeros	Construct	All-record R^2 overstates performance on needed records (0.87 against 0.63)	Both reported, Section 6.1.3
Deployed XGBoost regressor differs from the notebook run	Reproducibility	Shipped amounts differ slightly from the reported ones	in Colab
Regressor re-scored with an independent tree evaluator because XGBoost was not installable	Internal	Small risk that the evaluator differs from XGBoost	Results consistent; confirm in Colab
XGBoost and random forest regressor unseeded; library versions unpinned	Reproducibility	A rerun may give different artefacts	Seeds and pins proposed, Section 9.4
pH not sensed; growth stage and variety entered by hand	Construct	Input errors pass straight to the mode	Reported, D4

Stage label mismatch between screen and encoder	Internal	2 of 5 stages fail at the API	Reported, D1
No telemetry history	External	No trend or long-term validation possible	Reported, D2
No structured user study; informal feedback only	External	Usability and trust unproven	Reported, Section 6.3
Latency measured off the production server; server unreachable during write-up	External	End-to-end time not verified in the field	Reported, Section 6.2

Two further limitations are qualitative. The model has learned the labelling rule of this dataset, not crop physiology, so its decisions are only as good as the agronomic judgement behind the labels. And the recommendation ends at a screen: whether it changes what a farmer applies, and whether that changes yield, has not been measured.

7.6 Implications

Technical. Three practices generalise: report a regressor on the records where its output is used, not only on all records; check every categorical value the client can send against the encoder the model was trained with; and reload and re-score the shipped artefact rather than trusting the notebook's printout. The last two found the only functional defects in this study.

Agronomic. If the model mainly reproduces DoA nutrient thresholds, the most valuable next step is better labels: agronomist-confirmed decisions, separate N, P and K doses, and a stage schedule that matches DoA guidance. The inputs that today carry no weight (pH, moisture, temperature) should be used for explicit warnings until labelled data shows how they should change the dose.

Policy. A device that tells farmers when not to apply fertilizer supports DoA's efforts to cut fertilizer waste and reduces exposure to import shortages, provided the advice is validated in the field before it is promoted at scale.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

Value proposition. For a paddy farmer, the component answers one question at the moment it matters: does this field need fertilizer now, and how much? It measures the soil instead of relying on a calendar, and it almost never recommends fertilizer that is not needed, so its first value is money not spent and fertilizer not wasted.

Target users. Primary users are small and medium-scale paddy farmers, with an initial focus on Kurunegala district. Secondary users are agricultural instructors and officers who advise several farmers and can use the readings to prioritise field visits. Indirect beneficiaries are fertilizer suppliers, who gain a picture of demand by area and stage, and the DoA, which gains field-level evidence for its guidance.

8.2 Market and Competitive Position

The competitive frame has three categories. Calendar-based DoA schedules are free and trusted but take no account of the field's actual condition. IoT monitoring products and research systems [7], [12] measure the field but stop at the reading. Laboratory soil testing gives an accurate one-off picture but takes days and is not repeated during the season. This component competes on decision per reading: it turns a reading taken today into advice today. The technique itself is reproducible, so the durable advantage is the integrated device-to-phone workflow, the local varieties and stages, and the labelled field data that accumulates with use.

8.3 Business or Sustainability Model

The proposal costs the whole group system, not individual components, at LKR 75,000 for design, development, testing, deployment and maintenance (Table 8.1). The soil station's bill of materials is not part of that figure.

Table 8.1. Budget for the whole research system (LKR).

Item	Amount (LKR)
Design	15,000
Development	25,000
Testing	10,000
Deployment	15,000
Maintenance	10,000

The commercialisation path in the proposal has three steps: a pilot in selected Yala season fields to demonstrate savings; then distribution as a subscription, a device-and app bundle, or through agricultural cooperatives that share one station across neighbouring fields; and later, partnerships with fertilizer suppliers and government agencies. Sharing a station through a cooperative is the most realistic route for smallholders, because the device cost is spread across many fields

Sustainability. The software stack is open source, so it can be maintained without a proprietary vendor. The models can be retrained and re-issued as new artefacts without touching the device firmware. The main recurring costs are connectivity, hosting and sensor replacement; NPK probes in wet paddy soil need periodic cleaning and calibration, which a cooperative or extension officer could provide.

8.4 Scalability Intellectual Property and Compliance

Scalability. Each station writes to its own node and the prediction service holds no per field state, so adding fields is a matter of devices and nodes, not of redesign. Moving beyond a pilot requires the per-device history store (D2), HTTPS and authentication (D6), and a hosting arrangement with guaranteed uptime.

Intellectual property. The project claims no patents. Protectable assets are the integrated design, the trained artefacts and the labelled field dataset. Open-source libraries (scikit-learn, XGBoost, Flask, Flutter) are used under their licences.

Compliance. The component stores soil data, not personal data. The application's user records do contain personal data and are handled by the shared login component; their current storage in Firebase must be secured before public release (Section 5.4).

8.5 Contribution to the Sustainable Development Goals

The component contributes to two goals directly. SDG 2, Zero Hunger [36], is served by helping smallholders apply the nutrients their crop needs at the right time, protecting yield. SDG 12, Responsible Consumption and Production [37], is served by cutting unnecessary fertilizer use and the nutrient run-off that follows it. Indirectly, a measured, field-level view of nutrient status helps farmers cope with fertilizer supply shocks, because limited stock can be directed to the fields that need it most.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis set out to determine whether a low-cost field device could turn soil readings into a clear fertilization decision for Sri Lankan paddy farmers. The answer is a qualified yes. The implemented component reads soil moisture, temperature, humidity and N, P and K in the field, delivers them to a phone every 5 seconds, and returns "apply / do not apply" with an amount in kg/ha. Its classifier decides need with 0.9400 accuracy on unseen records, recommends unneeded fertilizer in only 3 of 4,800 cases, and is exactly reproducible from the dataset. The qualified part matters equally. The decision is driven almost entirely by N, P and K, so the soil context the proposal emphasised has little effect. The amount is indicative rather than prescriptive, with a typical error of 4.6 kg/ha on the records that need fertilizer and a stage pattern that does not follow DoA guidance. Two of five growth stages fail at the API, readings are not stored as history, and the application has not been tested in a structured study. These points bound the claims, and they are listed in Table 7.1 so that the next iteration inherits them as a work list rather than rediscovering them as defects.

9.2 Achievement of Objectives

Table 9.1 summarizes the status of the five individual objectives against the evidence presented in this thesis.

Table 9.1. Achievement of individual objectives.

Objective	Status	Evidence	Residual qualification
IO1 – Field sensing	Achieved	Soil station deployed in the field; live reading in Firebase (Section 5.2)	pH and EC not sensed; firmware not yet in the repository
IO2 – Transmission and storage	Partly achieved	Firebase node read every 5 s by the app (Table 5.2)	Latest reading only; no history or identifiers (D2)
IO3 – Need classification	Achieved	Random forest 0.9400 accuracy, CV 0.9369 ± 0.0029 (Section 6.1.2)	Recall capped near 92 per cent by unexplained labels
IO4 – Amount estimation	Partly achieved	Cascade within ±5 kg/ha for 70.2 per cent of records (Section 6.1.3)	R^2 0.63 on needed records; better regressor not deployed
Objective	**Status**	**Evidence**	**Residual qualification**

IO5 – Delivery to the farmer	Partly achieved	/iot/predict and the Advisor screen (Sections 4.4, 5.2)	Stage-label defect D1; server unreachable at write-up; English only

9.3 Individual Research and Product Contributions

The product contribution is a working fertilization feature inside the group's paddy DSS: a field soil station, live telemetry on the farmer's phone, and a reproducible two-stage recommendation service. The research contributions are the field-to-decision pipeline for local varieties and growth stages, an error analysis that places the model's ceiling in its labels rather than its algorithm, evidence on which soil context actually influences the decision, and an independent verification that confirmed the classifier and exposed the regressor and contract defects.

9.4 Future Directions

The following work follows directly from the findings, in priority order.

1. **Fix the stage contract (D1).** Send panicle_initiation and grain_filling from the dropdown, and add a test that passes every client value through the encoder. This is a one-line change that restores two of five stages.
2. **Store history (D2).** Write each reading to a time-keyed list per device as well as to the latest node, so that trends, audits and future training data become possible.
3. **Deploy the better regressor and make export reproducible.** Replace the XGBoost regressor with the random forest regressor, train it only on records that need fertilizer, set seeds, pin library versions, and save the models with their versions and metrics.

4. **Improve the labels.** Ask agronomists to confirm a sample of labels, split the amount into N, P and K doses, and align the stage schedule with DoA guidance; then retrain.
5. **Add explicit warnings.** Raise rule-based warnings for pH, moisture and temperature outside the DoA ranges (Appendix A), alongside the model's decision, and flag implausible readings such as N = 0.
6. **Secure and harden the service (D5, D6).** Serve over HTTPS behind a domain, add authentication, return proper HTTP status codes, and apply Firebase security rules.
7. **Run a structured field study.** Test the screen with at least 10 farmers against the proposal's 80 per cent task-completion target, add Sinhala and Tamil, and compare fertilizer use and yield between advised and unadvised plots over one season.

REFERENCES

[1] M. Perera, D. Jayasundara, and P. Samarasinghe, "IoT-Enabled Smart Solution for Rice Disease Detection and Yield Prediction in Sri Lanka," SLIIT Research Repository, 2023. [Online]. Available: https://rda.sliit.lk/handle/123456789/3524

[2] S. Wijekoon, H. Karunaratne, and P. Silva, "Wireless Sensor Network Based Smart Irrigation System for Paddy Cultivation in Sri Lanka," Proc. IEEE ICIAFS, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9779817

[3] N. Gunawardena and H. Karunaratne, "IoT Based Smart Agriculture Monitoring System for Sri Lankan Farmers," Proc. IEEE Industrial Informatics Conf., 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9456732

[4] R. Jayasinghe, K. Fernando, and M. Silva, "Low Cost IoT Solution for Precision Paddy Farming in Sri Lanka," University of Moratuwa Engineering Research, 2020. [Online]. Available: http://dl.lib.mrt.ac.lk/handle/123/15432

[5] P. Rathnayake, S. Bandara, and T. Perera, "Smart Irrigation Control using IoT for Rice Fields in Dry Zone Sri Lanka," Proc. IEEE TENCON, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9693281

[6] D. Senanayake, W. Kumara, and L. Peris, "Soil Moisture Monitoring System for Paddy Cultivation using IoT," SLIIT Conference Proceedings, 2022. [Online]. Available: https://rda.sliit.lk/handle/123456789/2987.

[7] K. Fernando, P. De Silva, and H. Wijesinghe, "Precision Agriculture using IoT Sensors for Sri Lankan Paddy Fields," IEEE Xplore, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9154210.

[8] H. Peris, A. Jayawardena, and R. Gunasekara, "Real Time Farm Monitoring System for Sri Lanka using IoT," International Journal of Smart Agriculture, 2021. [Online]. Available: https://ijsar.org/article/view/112.

[9] W. Abeysinghe, D. Nissanka, and K. Pathirana, "LoRa Based Agriculture Monitoring System for Paddy Fields," Proc. IEEE ICCC, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9887654.

[10] A. Dissanayake, M. Hettiarachchi, and S. Perera, "IoT Based Decision Support System for Sri Lankan Farmers," University of Peradeniya Research Papers, 2023. [Online]. Available: https://ir.lib.pdn.ac.lk/handle/123456789/16234.

[11] T. Bandara, R. Silva, and P. Karunarathna, "Smart Farming Platform for Rice Cultivation using IoT," Proc. IEEE Region 10 Conf., 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9491233.

[12] M. Wickramasinghe, L. Kumara, and N. Perera, "Cloud Connected Sensor System for Paddy Field Monitoring," Proc. IEEE ICTer Sri Lanka, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9325412.

[13] L. Kumara, S. Jayasinghe, and H. Perera, "Automated Irrigation and Fertilizer Suggestion System using IoT," SLIIT Undergraduate Research, 2022. [Online]. Available: https://rda.sliit.lk/handle/123456789/3112.

[14] Department of Agriculture Sri Lanka, "Rice Fertilizer Recommendation Guide," Rice Research Institute, 2020. [Online]. Available: https://www.doa.gov.lk

[15] B. A. M. Azmi, "Optimum Fertilizer Application Time in Lowland Rice in Sri Lanka," Tropical Agricultural Research, vol. 30, no. 3, 2019. [Online]. Available: https://tar.sljol.info.

[16] R. Nugaliyadde, S. Jayawardena, and P. Senanayake, "Nutrient Management in Rice Cultivation in Sri Lanka," Department of Agriculture, 2018. [Online]. Available: https://www.doa.gov.lk

[17] S. P. Withanage, "Effect of Nitrogen Timing on Rice Yield in Sri Lanka," University of Ruhuna Journal, 2017. [Online]. Available: https://ir.lib.ruh.ac.lk

[18] R. M. M. Rathnayake, "Fertilizer Scheduling for Paddy under Irrigated Conditions," Peradeniya Agricultural Journal, 2019. [Online]. Available: https://ir.lib.pdn.ac.lk

[19] H. M. Somapala, "Rice Crop Nutrient Requirement in Sri Lankan Soils," DOA Research Bulletin, 2016. [Online]. Available: https://www.doa.gov.lk

[20] FAO, "Best Practices for Rice Fertilization in South Asia," Food and Agriculture Organization, 2021. [Online]. Available: https://www.fao.org.

[21] IRRI, "Timing of Nitrogen Application in Rice Production," International Rice Research Institute, 2018. [Online]. Available: https://www.irri.org.

[22] J. K. Wijebandara, "Yield Improvement of Paddy through Proper Fertilizer Timing," Sri Lanka Journal of Agriculture, 2020. [Online]. Available: https://slja.sljol.info.

[23] P. B. Karunatilake, "Soil Fertility Management in Sri Lankan Paddy Lands," University of Peradeniya, 2015. [Online]. Available: https://ir.lib.pdn.ac.lk

[24] IRRI, "Rice Crop Growth Stages and Fertilizer Timing," International Rice Research Institute, 2019. [Online]. Available: https://knowledgebank.irri.org.

[25] Department of Agriculture Sri Lanka, "Paddy Cultivation Calendar and Fertilizer Schedule," Rice Research Institute Bathalagoda, 2022. [Online]. Available: https://www.doa.gov.lk

[26] J. K. Nayak et al., "IoT in Agriculture: Applications and Challenges," IEEE Internet of Things Journal, vol. 7, no. 6, pp. 4895–4906, 2020.

[27] S. R. Mishra et al., "Precision Agriculture using IoT and Machine Learning," IEEE Access, vol. 9, pp. 121–135, 2021.

[28] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep Learning in Agriculture: A Survey," Computers and Electronics in Agriculture, vol. 147, pp. 70–90, 2018

[29] FAO, "Smart Agriculture and Precision Farming," Food and Agriculture Organization, 2021

[30] IRRI, "Nutrient Management in Rice Systems," International Rice Research Institute, 2019.

[31] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[32] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[33] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," Journal of Management Information Systems, vol. 24, no. 3, pp. 45–77, 2007.

[34] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004.

[35] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[36] United Nations DESA, "Goal 12: Ensure sustainable consumption and production patterns." [Online]. Available: https://sdgs.un.org/goals/goal12.

GLOSSARY

Cascade (two-stage model). A classifier decides whether fertilizer is needed; a regressor estimates the amount only when it is.

Confusion matrix. A table of correct and incorrect predictions by actual and predicted class.

Cross-validation. Repeated splitting of the data into training and validation folds to estimate how stable a result is.

False negative (missed need). A field that needs fertilizer but is told it does not.

False positive (unneeded recommendation). A field that does not need fertilizer but is told it does.

Feature importance. The share of a random forest's splitting gain contributed by each input, summing to one.

Growth stage. One of the five DoA stages used here: germination, tillering, panicle initiation, heading and grain filling.

Label encoder. A mapping from text categories, such as a rice variety, to the integers a model uses.

MAE / RMSE. Mean absolute error and root mean squared error, here in kg/ha. R^2. The share of variance in the target explained by the model; 1 is perfect, 0 is no better than the mean.

Soil station. The ESP32 device in the field that reads the soil and air sensors and writes to Firebase.

Telemetry. The stream of readings sent from the soil station to the cloud.

Yala season. Sri Lanka's minor cultivation season, roughly April to September.

APPENDICES

Appendix A – DoA reference values used as expected behaviour

A.1 Soil sensor reference values, Yala season (DoA, 2022, as given in the project proposal).

Parameter	Optimal range	Critical thresholds
Soil moisture (%)	60–80	below 40 / above 90
Soil pH	5.5–6.5	below 5 / above 7
N (mg/kg)	25–35	below 15 / above 50
P (mg/kg)	15–25	below 10 / above 40
K (mg/kg)	120–200	below 80 / above 300
Temperature (°C)	28–32	below 20 / above 38
Humidity (%)	70–85	below 50 / above 95

A.2 2 Fertilizer dosage by growth stage, Yala season (DoA, 2022, as given in the project proposal).

Table A.1. Full threshold sweep (5,000 records).

Growth stage	N (kg/ha)	P (kg/ha)	K (kg/ha)	Total (kg/ha)
Germination	20	10	15	45
Tillering	25	12	20	57
Panicle initiation	20	10	15	45
Heading	15	5	10	30
Grain filling	10	0	5	15

Appendix B – DoA reference values used as expected behaviour

B.1 Verification environment. Python 3.13 with scikit-learn 1.9.1, numpy 2.5.3 and pandas, on a 2-core x86 Linux machine. XGBoost could not be installed there, so the deployed regressor's 200 trees were read directly from the model's serialised booster (UBJSON format, XGBoost 3.2.0) and evaluated with a short tree-walking script that applies XGBoost's split rule (go left when the value is less than the split condition) and adds the base score of 19.97

B.2 Bounded values in the dataset

Column	Minimum	Records at minimum	Maximum	Records at maximum
soil_moisture (%)	30.0	250	95.0	582
soil_temp (°C)	24.0	601	38.0	99
soil_ph	4.5	355	7.5	362
nitrogen	5.0	256	60.0	12
phosphorus	5.0	1,254	45.0	5
potassium	50.0	157	300.9	1
humidity (%)	50.0	156	95.0	531

Appendix C – Academic integrity and AI use disclosure

This appendix is provided in accordance with the module policy on generative artificial intelligence, which requires disclosure of any AI assistance while leaving responsibility with the candidate.

Tools used. A large language model assistant made by Anthropic, was used to organise and structure the candidate’s written content into the thesis format and to improve the flow and wording of the drafts.

Work performed by the candidate personally. The candidate wrote the original ideas and content of this thesis; designed and built the soil station; collected the field data; wrote and ran the training notebook; integrated the models into the API and the mobile screen; gathered farmer feedback; and made every interpretation, comparison and conclusion presented. The candidate takes responsibility for the originality of the claims.

Verification. Every number quoted in this thesis is traceable to a project artefact: the collected dataset, the training notebook outputs, or the verification results reported in Appendix B. The candidate checked each reported value against these sources.

Declaration. I declare that the use of AI assistance has been disclosed above, that it has been used in accordance with the module policy, and that I remain fully responsible for the accuracy, originality, citations, code, data, analyses and conclusions presented in this thesis.

Signature: ___ **_____ · Date: ___2026/10/11_____**